

HEIMDALL Development Constitution

Version 1.0

Status: Frozen (Baseline)

Document Information

Field	Value
Project	HEIMDALL
Version	1.0
Status	Frozen
Document Type	Project Constitution
Applies To	Entire Project
Last Updated	Version 1.0
Approval Authority	Project Owner

Revision History

Version	Description
1.0	Initial frozen development constitution

Future revisions (v1.1, v2.0, etc.) shall update this table.

1. Vision

HEIMDALL is a **Modular Embedded System Supervision Framework (ESSF)** designed to monitor, verify, and supervise embedded communication and software execution.

Unlike a traditional watchdog timer, HEIMDALL performs intelligent supervision through configurable policies, heartbeat monitoring, challenge-response verification, protocol-aware monitoring, fault classification, and recovery management.

The long-term objective is to create a reusable supervision IP suitable for:

- Embedded systems
- Automotive ECUs
- Industrial controllers
- Aerospace systems
- Robotics
- IoT devices
- Research and education

Version 1 establishes the architectural foundation upon which future protocol modules and advanced supervision features shall be built.

2. Mission Statement

The objective of HEIMDALL is **not merely to produce functioning code.**

The project shall demonstrate professional engineering practices across the complete development lifecycle:

- Requirements Engineering
- Architecture Design
- Interface Design
- Embedded Software
- RTL Design
- Verification
- Documentation
- Version Control
- Continuous Improvement

Every engineering decision shall prioritize maintainability, correctness, reproducibility, and verification.

3. Project Objectives

HEIMDALL shall:

- Provide protocol-aware supervision.
- Detect communication anomalies.
- Detect execution anomalies.
- Classify failures.
- Provide configurable recovery mechanisms.
- Support multiple embedded communication interfaces.
- Maintain equivalent Embedded C and RTL behavior.

- Be fully simulation driven.
 - Use only free development tools.
 - Produce a professional GitHub portfolio project.
-

4. Scope

Version 1

Version 1 shall implement:

Core Framework

- Generic supervision engine
- Heartbeat monitoring
- Challenge-response verification
- Fault detection
- Fault classification
- Recovery manager
- Event logger
- Configuration manager
- Parameterized timing
- Embedded C reference model
- Synthesizable Verilog RTL
- Complete verification environment

Supported Communication Interfaces

Version 1 shall include supervision support for:

- SPI
- UART
- I²C
- GPIO

Each protocol shall be implemented as an independent module using the same supervision philosophy while allowing protocol-specific timing and monitoring behavior.

The architecture shall permit enabling or disabling protocol modules without affecting the remaining system.

Future Versions

Future versions may include:

Additional Interfaces

- CAN
- CAN-FD
- LIN
- USB
- Ethernet
- PCI Express
- SDIO
- eSPI
- MIPI
- JTAG
- SWD
- SMBus
- PMBus
- 1-Wire
- Modbus

Advanced Features

- TPM integration
- Secure Boot integration
- Measured Boot supervision
- Multi-core supervision
- Distributed supervision
- Hierarchical supervision
- Adaptive timeout policies
- Runtime profile switching
- Secure firmware authentication
- Cryptographic challenge engines

Deployment

- FPGA reference platform
- ASIC synthesis flow
- Linux driver
- RTOS integration
- Browser dashboard
- CI/CD pipeline
- Hardware evaluation board

Version 1 shall not be architecturally constrained by future additions.

5. Development Philosophy

The project shall follow an engineering-first workflow.

Every feature shall follow the same lifecycle:

Requirements



Architecture



Specification



Interfaces



Implementation



Verification



Documentation



Git Commit



Integration

Implementation shall never precede architecture.

6. Engineering Principles

The project shall prioritize:

1. Correctness over speed.

2. Specification over implementation.
 3. Verification over assumptions.
 4. Modularity over monolithic design.
 5. Readability over optimization.
 6. Reproducibility over convenience.
 7. Incremental development over large unverified changes.
 8. Deterministic behavior.
 9. Open-source tooling.
 10. Professional engineering discipline.
-

7. Source of Truth

Document precedence shall be:

1. Development Constitution
2. Requirements
3. Architecture
4. Register Map
5. Protocol Specification
6. Interface Definitions
7. Embedded C Reference
8. RTL
9. Documentation Examples

When conflicts occur, the higher-priority document shall prevail.

8. Roles and Responsibilities

Project Owner

Responsible for:

- Executing commands
 - Creating files
 - Running simulations
 - Reviewing outputs
 - Uploading to GitHub
 - Managing repository
-

ChatGPT

Acts as:

- System Architect
- Technical Lead
- Verification Lead
- Documentation Engineer
- Code Reviewer
- Debugging Assistant

Responsibilities include:

- Architecture
- Specifications
- Interfaces
- Design reviews
- Documentation
- Consistency checks
- Debugging
- Verification planning

ChatGPT owns architectural decisions.

Gemini

Acts as:

Implementation Assistant.

Responsibilities:

- Embedded C generation
- RTL generation
- Boilerplate code
- Testbench generation
- Refactoring

Gemini shall never redefine architecture.

Every Gemini-generated artifact shall be reviewed before acceptance.

9. AI Workflow

ChatGPT

↓

Architecture

↓

Specification

↓

Gemini Prompt

↓

Gemini Implementation

↓

ChatGPT Review

↓

User Integration

↓

Simulation

↓

Verification

↓

Documentation

↓

Git

No generated code shall bypass review.

10. Development Environment

Editor:

- VS Code

Purpose:

Editing only.

The editor shall not be considered part of the build system.

Primary Environment:

PowerShell

PowerShell shall be used for:

- Project creation
- Directory management
- Compilation
- Simulation
- Verification
- Git
- GitHub
- Cleanup
- Automation

PowerShell is the official development environment.

11. Toolchain

Mandatory

- VS Code
- PowerShell
- GCC
- Icarus Verilog
- GTKWave
- Verilator
- Git
- GitHub

Optional

- Gemini
- GitHub Copilot
- draw.io
- Canva

No proprietary FPGA IDE shall be required.

12. Repository Structure

```
HEIMDALL/  
|  
├─ docs/  
|  
├─ firmware/  
|  
├─ rtl/  
|  
├─ tb/  
|  
├─ scripts/  
|  
├─ build/  
|  
├─ waves/  
|  
├─ assets/  
|  
├─ examples/  
|  
├─ .github/  
|  
├─ README.md  
|  
├─ LICENSE  
└─ .gitignore
```

This structure shall remain stable throughout Version 1.

13. Documentation Structure

```
docs/  
  
01_project_overview.md  
  
02_requirements.md  
  
03_architecture.md  
  
04_register_map.md  
  
05_protocol_specification.md  
  
06_module_interfaces.md  
  
07_firmware_design.md  
  
08_rtl_design.md  
  
09_verification_plan.md  
  
10_test_results.md  
  
11_future_work.md
```

Documentation shall evolve alongside implementation.

14. Firmware Structure

```
firmware/  
  
main.c  
  
configuration.h  
  
heartbeat.c  
  
heartbeat.h  
  
challenge.c  
  
challenge.h
```

```
fault_classifier.c
fault_classifier.h
recovery.c
recovery.h
event_logger.c
event_logger.h
spi_supervisor.c
uart_supervisor.c
i2c_supervisor.c
gpio_supervisor.c
```

15. RTL Structure

```
rtl/
heimdall_top.v
configuration.v
heartbeat_engine.v
challenge_engine.v
fault_classifier.v
timeout_engine.v
recovery_manager.v
event_logger.v
spi_supervisor.v
uart_supervisor.v
```

```
i2c_supervisor.v  
gpio_supervisor.v
```

Every module shall have one clearly defined responsibility.

16. Verification Structure

```
tb/  
heartbeat_tb.v  
challenge_tb.v  
timeout_tb.v  
fault_tb.v  
spi_tb.v  
uart_tb.v  
i2c_tb.v  
gpio_tb.v  
heimdall_tb.v
```

Every RTL module shall have an independent testbench before system integration.

17. Coding Standards

Verilog

- Verilog-2001
- Synthesizable RTL
- Explicit reset
- Named parameters
- Consistent indentation
- No vendor primitives

- One module per file
-

Embedded C

- ANSI C / C99
 - Modular APIs
 - No dynamic memory
 - Portable implementation
 - Header/source separation
 - Consistent naming
-

18. Module Development Workflow

Every module shall follow:

1. Define purpose.
2. Define requirements.
3. Design architecture.
4. Freeze interface.
5. Generate Gemini prompt.
6. Implement.
7. Review.
8. Compile.
9. Lint.
10. Simulate.
11. Verify.
12. Update documentation.
13. Commit.

No step shall be skipped.

19. Deliverables Per Module

For every module, ChatGPT shall provide:

- Purpose
- Functional description
- Design rationale
- Inputs
- Outputs
- Parameters
- Internal architecture

- State machine (if required)
 - Timing behavior
 - Gemini prompt
 - Review checklist
 - PowerShell commands
 - Build commands
 - Simulation commands
 - Expected outputs
 - Common bugs
 - Verification checklist
 - Suggested Git commit
 - Required documentation updates
-

20. Simulation Workflow

PowerShell

↓

Icarus Verilog

↓

vvp

↓

dump.vcd

↓

GTKWave

↓

Verification

Waveforms shall be retained as project evidence.

21. Verification Philosophy

A module is considered complete only after:

- Successful compilation
 - Successful linting
 - Successful unit testing
 - Successful waveform verification
 - Documentation update
 - Git commit
-

22. Git Workflow

Development shall use small logical commits.

Typical progression:

- Architecture
- Configuration
- Heartbeat
- Challenge
- Timeout
- Fault Classification
- Recovery
- Protocol Supervisors
- Integration
- Verification
- Documentation

Large unreviewed commits are discouraged.

23. PowerShell Policy

Every development action shall be executable from PowerShell.

Typical activities include:

- Creating files
- Compiling firmware
- Compiling RTL
- Running simulations
- Viewing waveforms

- Linting
- Cleaning builds
- Running helper scripts
- Git operations

VS Code is used solely for editing.

24. Automation

Version 1 may include helper scripts:

```
scripts/  
  
build.ps1  
  
run.ps1  
  
lint.ps1  
  
clean.ps1  
  
test.ps1
```

These scripts shall automate repetitive development tasks.

25. Definition of Done (Module)

A module is complete only when:

- Requirements implemented
 - Interface frozen
 - Code reviewed
 - Compilation successful
 - Lint clean
 - Testbench passes
 - Waveforms verified
 - Documentation updated
 - Commit completed
-

26. Definition of Done (Project)

HEIMDALL Version 1 is complete only when:

- Constitution finalized
 - Documentation complete
 - Architecture complete
 - Register map finalized
 - Firmware complete
 - RTL complete
 - SPI supervisor verified
 - UART supervisor verified
 - I²C supervisor verified
 - GPIO supervisor verified
 - Integration successful
 - All simulations pass
 - Waveforms archived
 - README completed
 - Repository organized
-

27. Out of Scope

The following are intentionally excluded from Version 1 implementation:

- Physical FPGA deployment
- ASIC fabrication
- Production PCB
- Linux kernel drivers
- Desktop GUI
- Browser visualization
- Cloud deployment
- Hardware evaluation boards

These belong to future releases.

28. Change Control

Architectural changes shall not occur after implementation begins unless:

1. The Project Owner explicitly approves the change.
2. This Constitution is revised.
3. The revision history is updated.

Bug fixes do not require constitutional revision.

29. Long-Term Vision

HEIMDALL is intended to evolve into a reusable supervision framework supporting multiple communication protocols, multiple deployment targets, and multiple verification environments while preserving a stable, modular architecture.

Every new capability shall extend the framework through reusable modules rather than modifying existing architectural foundations.

30. Final Declaration

This document serves as the governing engineering specification for **HEIMDALL Version 1**.

All architectural decisions, implementation activities, verification efforts, documentation, and repository organization shall conform to this Constitution unless a formally approved revision supersedes it.

This document is the project's single source of truth and shall remain stable throughout Version 1 development.